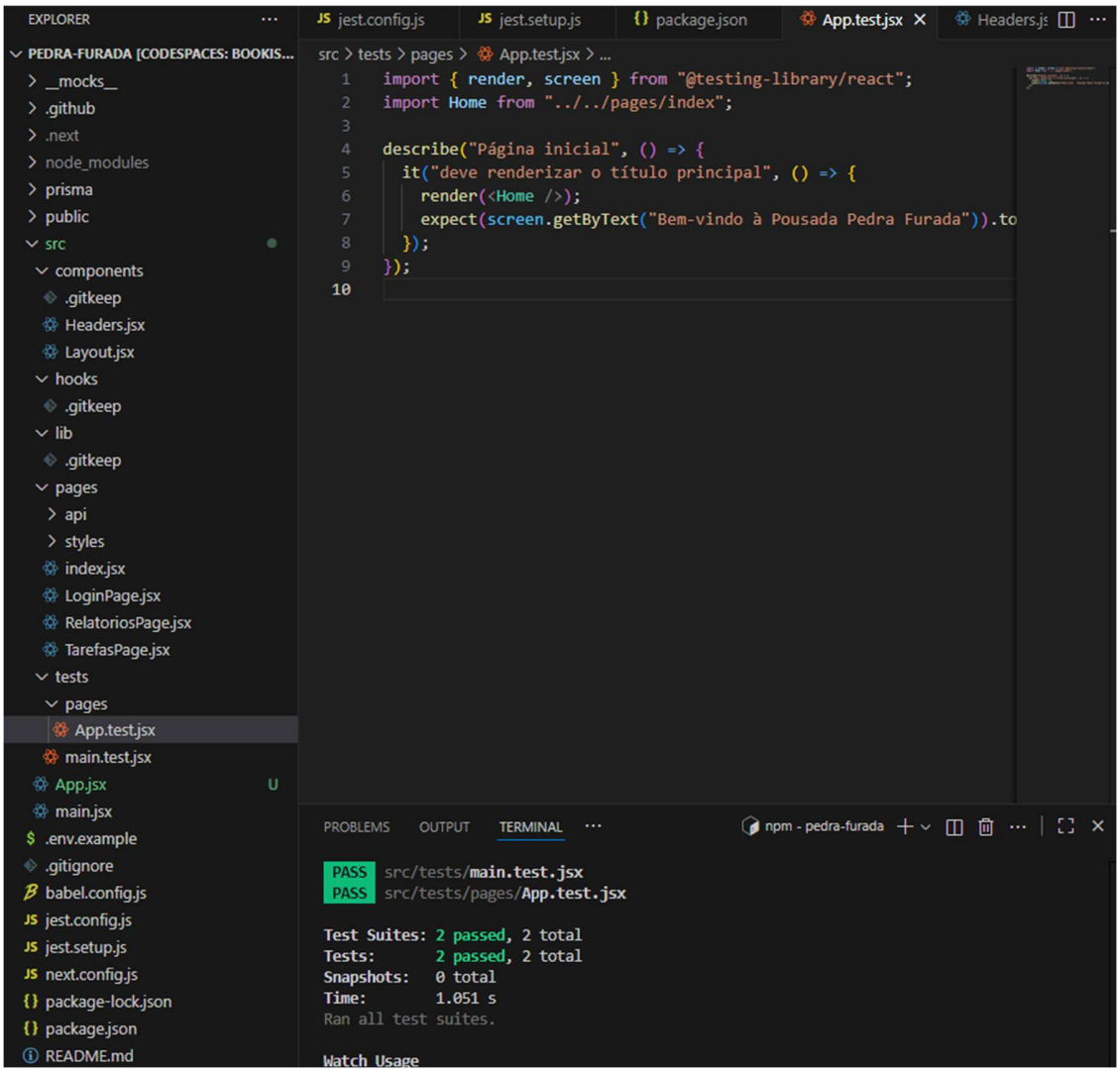
Sumário

1. Refatorando antes de prosseguir: 2
2. Implementando as rotas das páginas no arquivo App.jsx: 3
   2.1. Testando o roteamento da SPA – App.test.jsx: 4
   2.2. ReferenceError: TextEncoder is not defined: 5
   2.3. Um erro para cada rota: 5
3. Implementando o Layout.test.jsx: 8
   3.1. Criando o Layout.test.jsx: 8
4. Adicionando o Layout.jsx e fazendo passar no teste: 9
5. Refatorando o Layout.jsx: 10
   5.1. Conteúdo do Header.jsx: 10
   5.2. Conteúdo do Footer.jsx: 10
   5.3. Conteúdo do Layout.jsx refatorado e compatível com o teste atual: 11
6. Após refatorar o Layout.jsx: 11
7. Testando o Header e Footer: 12
   7.1. Motivos para separar os testes: 12
   7.2. Teste Header.test.jsx: 12
   7.3. Teste Footer.test.jsx: 13
   7.4. Ajustando o Layout.test.jsx: 13

1. Refatorando antes de prosseguir:

Finalizada a montagem do ambiente, a configuração inicial e com os dois testes iniciais `main.test.jsx` e `App.test.jsx` executando, conforme print de tela abaixo, agora é hora de organizar o código, é hora de refatorar o código.



Analisando o código do arquivo `App.test.jsx` que está dentro do diretório `src/tests/pages/`, dá para notar que na verdade esse teste está testando a página `index.jsx` e não o roteamento das páginas.

Renomeie esse arquivo para um nome significativo, que no caso seria `index.test.jsx`, indicando que a página `index.jsx` é quem está sendo testada. Os dois testes `main.test.jsx` e `index.test.jsx` devem passar.

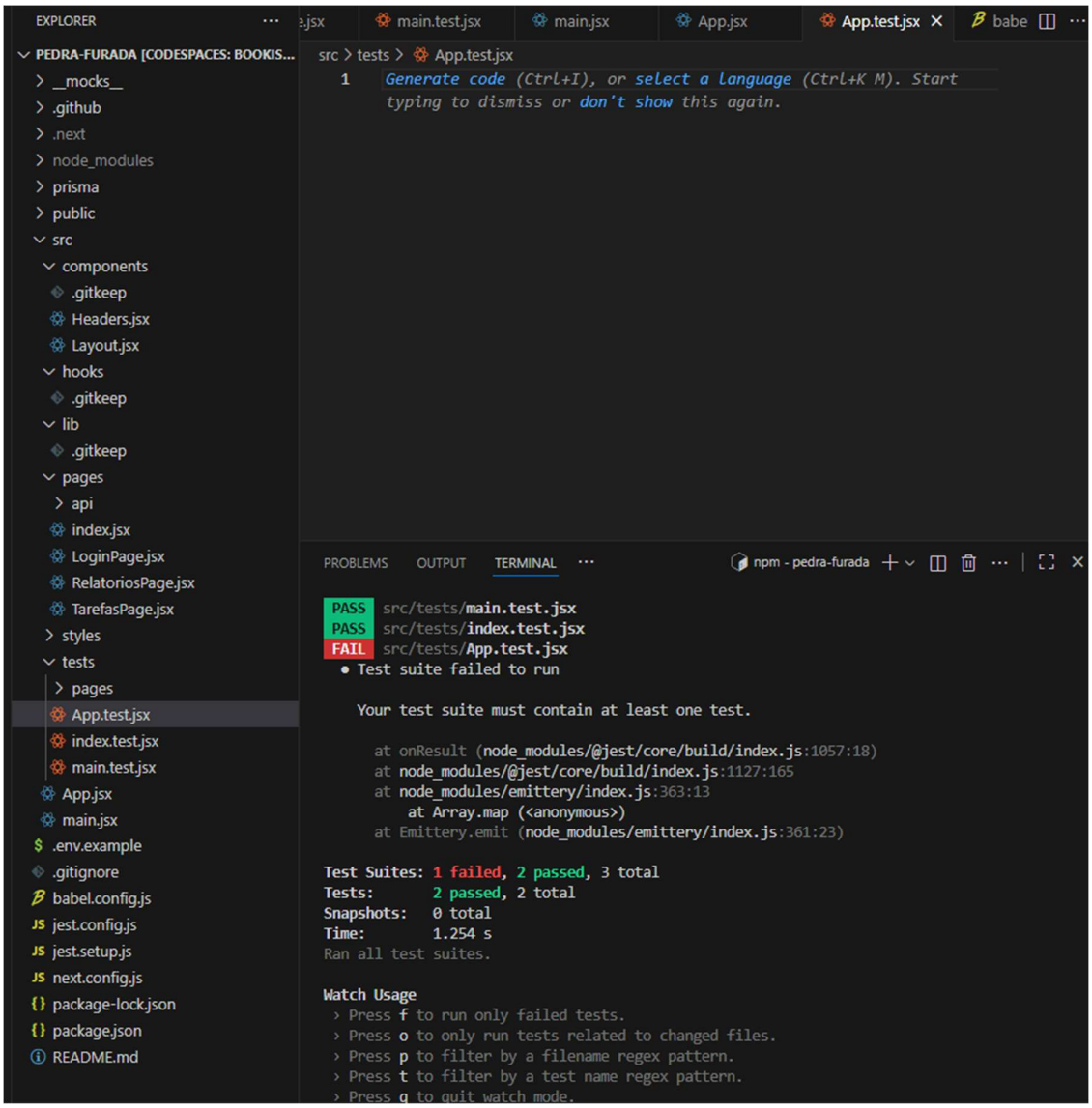
Proceder com o commit: "refactor: Ajustando os diretórios e renomeando os testes".

2. Implementando as rotas das páginas no arquivo App.jsx:

O arquivo `App.jsx`, componente raiz da SPA, é responsável por centralizar o roteamento entre as páginas e deve ser criado no caminho `/src/App.jsx`, no mesmo nível de `/src/main.jsx`, que atua como ponto de entrada da aplicação.

Seguindo o ciclo do TDD, o `App.jsx` será implementado após a criação do teste `/src/tests/App.test.jsx`, garantindo a transição da aplicação para uma SPA client-side pura, verificando se as rotas "/", "/login", "/tarefas", "/relatórios" e "/rota-invalida" definidas em `App.jsx` serão renderizadas corretamente.

Conforme indicado abaixo, ao criar o arquivo vazio `/src/tests/App.test.jsx`, a suíte de teste irá entender que um terceiro teste foi criado e ele irá falhar.



2.1. Testando o roteamento da SPA – App.test.jsx:

Esse teste deve validar o comportamento de roteamento definido no App.jsx, garantindo que 5 rotas no caso, sejam configuradas corretamente: "/", "/login", "/tarefas", "/relatórios" e "/rota-invalida" e renderizem corretamente o componente esperado.

Popule o arquivo App.test.jsx com o seguinte conteúdo:

```
import { render, screen } from "@testing-library/React";
import { MemoryRouter } from "React-router-dom";
import App from "../App";

// Mocka os componentes e páginas para testar apenas o roteamento
jest.mock("../components/Layout", () => ({ children }) => (
  <div data-testid="layout">{children}</div>));

jest.mock("../pages/LoginPage", () => () => <div>Login Page Mock</div>);
jest.mock("../pages/TarefasPage", () => () => <div>Tarefas Page Mock</div>);
jest.mock("../pages/RelatoriosPage",  ()  =>  ()  =>  <div>Relatórios  Page
Mock</div>);

describe("App.jsx – Roteamento principal da SPA", () => {
  it("deve redirecionar '/' para '/login'", () => {
    render(
      <MemoryRouter initialEntries={["/"]}><App /></MemoryRouter>
    );
    expect(screen.getByText("Login Page Mock")).toBeInTheDocument();
  });
  it("deve renderizar a página de login ao acessar '/login'", () => {
    render(
      <MemoryRouter initialEntries={["/login"]}><App /></MemoryRouter>
    );
    expect(screen.getByText("Login Page Mock")).toBeInTheDocument();
  });

  it("deve renderizar a página de tarefas ao acessar '/tarefas'", () => {
    render(
      <MemoryRouter initialEntries={["/tarefas"]}> <App /></MemoryRouter>
    );
    expect(screen.getByText("Tarefas Page Mock")).toBeInTheDocument();
  });

  it("deve renderizar a página de relatórios ao acessar '/relatorios'", () => {
    render(
      <MemoryRouter initialEntries={["/relatorios"]}>
        <App />
      </MemoryRouter>
    );
    expect(screen.getByText("Relatórios Page Mock")).toBeInTheDocument();
  });

  it("deve exibir 'Página não encontrada' ao acessar uma rota inválida", () => {
    render(
      <MemoryRouter initialEntries={["/rota-invalida"]}>
        <App />
      </MemoryRouter>
    );
    expect(screen.getByText("Páginanão encontrada")).toBeInTheDocument();
  });
});
```

O teste deve executar imediatamente.

2.2. ReferenceError: TextEncoder is not defined:

Depois que popular o arquivo `App.test.jsx` com as rotas a serem testadas, caso ocorra o erro ReferenceError: TextEncoder is not defined, isso indica que existe um problema no ambiente Node.js, onde o Jest é executado.

Esse erro acontece porque o React-router-dom (desde a versão 6.14+) depende internamente de APIs do navegador como TextEncoder e TextDecoder. porém essas classes não existem nativamente no Node.js e precisam ser importadas manualmente.

Para corrigir, adicione as linhas abaixo ao final do arquivo de configuração do Jest, `jest.setup.js`, conforme abaixo:

```
// Corrige erro "TextEncoder is not defined"
import { TextEncoder, TextDecoder } from "util";
if (!global.TextEncoder) global.TextEncoder = TextEncoder;
if (!global.TextDecoder) global.TextDecoder = TextDecoder;
```

Proceder com o commit: feat: Adicionando o teste de roteamento App.test.jsx e configurando o Jest

2.3. Um erro para cada rota:

Corrigido o ReferenceError, o teste deve subir 5 erros:

- `App.jsx — Roteamento principal da SPA › deve redirecionar '/' para '/login'`
- `App.jsx — Roteamento principal da SPA › deve renderizar a página de login ao acessar '/login'`
- `App.jsx — Roteamento principal da SPA › deve renderizar a página de tarefas ao acessar '/tarefas'`
- `App.jsx — Roteamento principal da SPA › deve renderizar a página de relatórios ao acessar '/relatorios'`
- `App.jsx — Roteamento principal da SPA › deve exibir 'Página não encontrada' ao acessar uma rota inválida`

Analisando os 5 erros, que são iguais, temos a mensagem:

"`Element type is invalid: expected a string (for built-in components) or a class/function (for composite components) but got: object. …`"

Esse erro indica que o Jest tentou renderizar o `<App />` mas o valor importado de "`../App`" não é uma função React, pode ser um objeto ou um "undefined".

É exatamente o que se espera encontrar no estágio "Red" do TDD.

O teste falha porque o componente App ainda não foi implementado.

Proceda populando o arquivo App.jsx com o conteúdo a seguir:

```
import React from "React";
import { Routes, Route, Navigate } from "React-router-dom";
import Layout from "./components/Layout";
import LoginPage from "./pages/LoginPage";
import TarefasPage from "./pages/TarefasPage";
import RelatoriosPage from "./pages/RelatoriosPage";
export default function App() {
  return (
    <Layout>
      <Routes>
        <Route path="/" element={<Navigate to="/login" />} />
        <Route path="/login" element={<LoginPage />} />
        <Route path="/tarefas" element={<TarefasPage />} />
        <Route path="/relatorios" element={<RelatoriosPage />} />
        <Route path="*" element={<h2>Página não encontrada</h2>} />
      </Routes>
    </Layout>
  );
}
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   npm - pedra-furada

  ● App.jsx — Roteamento principal da SPA › deve exibir 'Página não encontrada' ao acessar uma
rota inválida

    TestingLibraryElementError: Unable to find an element with the text: Páginanão encontrada.
This could be because the text is broken up by multiple elements. In this case, you can provide
 a function for your text matcher to make your matcher more flexible.

    Ignored nodes: comments, script, style
    <body>
      <div>
        <div
          data-testid="layout"
        >
          <h2>
            Página não encontrada
          </h2>
        </div>
      </div>
    </body>

      47 |       </MemoryRouter>
      48 |     );
    > 49 |     expect(screen.getByText("Páginanão encontrada")).toBeInTheDocument();
         |                   ^
      50 |   });
      51 | });
      52 |

      at Object.getElementError (node_modules/@testing-library/dom/dist/config.js:37:19)
      at node_modules/@testing-library/dom/dist/query-helpers.js:76:38
      at node_modules/@testing-library/dom/dist/query-helpers.js:52:17
      at node_modules/@testing-library/dom/dist/query-helpers.js:95:19
      at Object.getByText (src/tests/App.test.jsx:49:19)

 PASS  src/tests/index.test.jsx
 PASS  src/tests/main.test.jsx

Test Suites: 1 failed, 2 passed, 3 total
Tests:       1 failed, 6 passed, 7 total
Snapshots:   0 total
Time:        1.31 s
Ran all test suites.
```

Caso um dos 5 testes do `App.test.jsx` não passe e apresente o erro "`TestingLibraryElementError: Unable to find an element with the text: Páginanão encontrada`", tente analisar e corrigir esse erro antes de ler o próximo parágrafo.

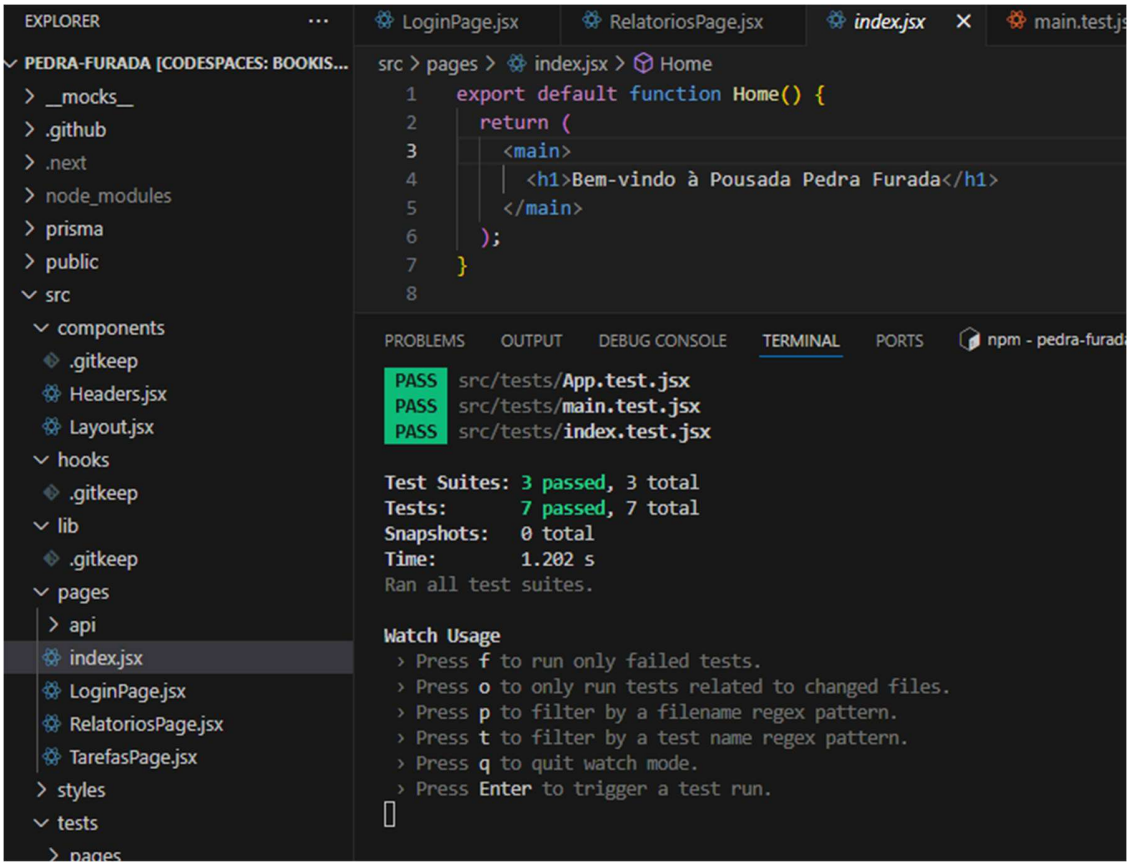
Esse problema aconteceu porque o arquivo de teste `App.test.jsx` tem um erro de digitação e o próprio erro está mostrando isso claramente.

Nesse caso é necessário corrigir o teste.

Corrija a linha:

De: "`expect(screen.getByText("Páginanão encontrada")).toBeInTheDocument();`"
Para: "`expect(screen.getByText("Página não encontrada")).toBeInTheDocument();`"

O teste das rotas deve passar.



Proceda com o commit: "feat: Adicionado teste de rotas e implementação das rotas."

3. Implementando o Layout.test.jsx:

Essa implementação tem como objetivo testar o componente `Layout.jsx`, que atua como container principal da SPA, responsável por gerenciar a estrutura visual das páginas, incluindo elementos fixos como cabeçalho e rodapé, e por renderizar corretamente o conteúdo dinâmico (children) de cada rota.

Em outras palavras, o `Layout.test.jsx` verifica se o `Layout.jsx` exibe corretamente o cabeçalho e o conteúdo das páginas, assegurando a consistência visual da aplicação.

3.1. Criando o Layout.test.jsx:

Abaixo segue o arquivo de teste `Layout.test.jsx` melhorado, com as seguintes características:

- O título do `describe` está contextualizado "Estrutura principal da aplicação"
- Cobertura de teste abrangendo Header, Conteúdo e Footer.
- Mais legível, pois dentro do `describe()` há um texto explicativo
- Um pouco mais semântico
- Regex no footer, pois utiliza `/Pousada Pedra Furada/i` para tolerar variações tipográficas (maiúsculas/minúsculas)

```
import { render, screen } from "@testing-library/react";
import Layout from "../components/Layout";
// Mocka o Header para isolar o teste do Layout
jest.mock("../components/Header", () => () => <header>Mock Header</header>);
describe("Layout.jsx – Estrutura principal da aplicação", () => {
  it("deve renderizar o Header, o conteúdo filho e o Footer", () => {
    render(
      <Layout>
        <p>Conteúdo interno</p>
      </Layout>
    );
    // Verifica se o cabeçalho foi renderizado
    expect(screen.getByText("Mock Header")).toBeInTheDocument();
    // Verifica se o conteúdo filho está presente
    expect(screen.getByText("Conteúdo interno")).toBeInTheDocument();
    // Verifica se o rodapé com o nome da pousada foi renderizado
    expect(screen.getByText(/Pousada Pedra Furada/i)).toBeInTheDocument();
  });
});
```

O novo teste deve falhar porque o `Layout.jsx` não foi implementado ainda.

```
PROBLEMS    OUTPUT    TERMINAL    ···                npm - pedra-furada

PASS  src/tests/App.test.jsx
PASS  src/tests/main.test.jsx
FAIL  src/tests/Layout.test.jsx
  ● Test suite failed to run

    Cannot find module '../components/Header' from 'src/tests/Layout.test.jsx'
```

Proceda com o commit: "test: adicionado teste de layout"

4. Adicionando o Layout.jsx e fazendo passar no teste:

Agora o teste `/src/tests/Layout.test.jsx` está apresentando o erro indicando que ele não conseguiu encontrar o módulo `/src/components/Header`.

Olhando a estrutura de diretórios, podemos ver que foi criado indevidamente um arquivo `/src/componentes/Headers.jsx`. Basta renomear esse arquivo para `Header.jsx`.

O teste continuará falhando, mas agora com o erro: "`TestingLibraryElementError: Unable to find an element with the text: /Pousada Pedra Furada/i`", conforme abaixo:

```
FAIL src/tests/Layout.test.jsx
  ● Layout.jsx – Estrutura principal da aplicação › deve renderizar o Header, o conteúdo fi
lho e o Footer

    TestingLibraryElementError: Unable to find an element with the text: /Pousada Pedra Fur
ada/i. This could be because the text is broken up by multiple elements. In this case, you
can provide a function for your text matcher to make your matcher more flexible.

    Ignored nodes: comments, script, style
    <body>
      <div>
        <div>
          <header>
            Mock Header
          </header>
          <main
            style="padding: 1rem;"
          >
            <p>
              Conteúdo interno
            </p>
          </main>
        </div>
      </div>
    </body>

      20 |
      21 |     // Verifica se o rodapé com o nome da pousada foi renderizado
    > 22 |     expect(screen.getByText(/Pousada Pedra Furada/i)).toBeInTheDocument();
```

Esse erro indica que o `Layout.test.jsx` encontrou o arquivo `Layout.jsx`, mas não encontrou um elemento que contenha o texto "`/Pousada Pedra Furada/i`".

Por enquanto vamos fazer o teste passar, ajustando o arquivo `Layout.jsx`, para ele entregar todos os elementos esperados pelo `Layout.test.jsx`.

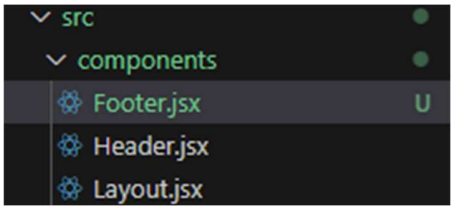
```
import React from "react";
import Header from "./Header";

export default function Layout({ children }) {
  return (
    <div>
      <Header />
      <main style={{ padding: "1rem" }}>{children}</main>
      <footer>
        <small>© 2025 Pousada Pedra Furada</small>
      </footer>
    </div>
  );
}
```

Proceda com o commit: “feat: Implementado o Layout.jsx e renomeado o arquivo Header.jsx”

5. Refatorando o Layout.jsx:

Agora que o teste passou, é hora de refatorar o `Layout.jsx`, lembrando que precisa criar os componentes `Header.jsx` e `Footer.jsx` no diretório `/src/components` e que o conteúdo dinâmico das páginas deve ser mantido no children .

Precisa ajustar o diretório `components` para ficar conforme abaixo:



5.1. Conteúdo do Header.jsx:

```
import React from "react";

export default function Header() {
  return (
    <header>
      <h1>Pousada Pedra Furada</h1>
    </header>
  );
}
```

5.2. Conteúdo do Footer.jsx:

```
import React from "react";

export default function Footer() {
  return (
    <footer>
      <small>© 2025 Pousada Pedra Furada</small>
    </footer>
  );
}
```

5.3. Conteúdo do Layout.jsx refatorado e compatível com o teste atual:

```
import React from "react";
import Header from "./Header";
import Footer from "./Footer";

export default function Layout({ children }) {
  return (
    <div className="layout">
      <Header />
      <main style={{ padding: "1rem" }}>
        {children}
      </main>
      <Footer />
    </div>
  );
}
```

Proceder com o commit: "refactor: Refatoração do Layout.jsx, Header.jsx e criação do componente Footer.jsx"

6. Após refatorar o Layout.jsx:

Após a validação do teste `Layout.test.jsx`, foi realizada a refatoração do componente `Layout.jsx`.

O objetivo foi separar responsabilidades dentro da estrutura da aplicação, mantendo o comportamento testado e garantindo que o código permanecesse limpo, modular e de fácil manutenção.

Durante essa refatoração, o componente `Layout.jsx`, que anteriormente concentrava todo o conteúdo da estrutura da página (cabeçalho, corpo e rodapé), foi dividido em três partes independentes:

1. **Header.jsx** — responsável pela renderização do cabeçalho da aplicação, contendo o título da *Pousada Pedra Furada*;
2. **Footer.jsx** — responsável por exibir o rodapé com as informações institucionais da pousada;
3. **Layout.jsx** — responsável por integrar os dois componentes anteriores e por manter o conteúdo dinâmico das páginas dentro da propriedade children, permitindo que cada rota da SPA injete seu próprio conteúdo no corpo principal.

Essa separação melhorou a organização, reuso e clareza semântica, além de preparar o projeto para futuras evoluções, como a inclusão de menus de navegação, barras laterais ou estilos personalizados, sem necessidade de alterar a lógica central.

É importante ressaltar que o teste `Layout.test.jsx` continuou passando, assegurando que a refatoração manteve o comportamento original do componente.

Agora o Header, o conteúdo interno e o `Footer` são renderizados correta e independentemente, preservando a consistência visual e a modularidade à aplicação.

7. Testando o Header e Footer:

Após a refatoração do componente `Layout.jsx`, os módulos `Header.jsx` e `Footer.jsx` passaram a existir como componentes independentes, cada um responsável por uma parte específica da interface da aplicação.

Essa modularização traz mais clareza e segmenta o layout da página, mas também demanda que a suíte de testes reflita essa nova estrutura, devendo ser individualizados, garantindo que cada componente seja verificado de maneira isolada, enquanto o `Layout.test.jsx` permanece focado em validar a integração entre eles.

7.1. Motivos para separar os testes:

- Responsabilidade Única (SRP): Assim como a refatoração dividiu o código em componentes autônomos (Layout, Header e Footer), a bateria de testes deve seguir o mesmo princípio. Cada componente precisa ter testes específicos para seu comportamento e conteúdo.
- Facilidade de Manutenção: Caso o header receba novos elementos, como menus, logotipo ou avatar, será possível ajustar apenas seu respectivo teste, sem afetar o `Layout.test.jsx`.
- Evolução Independente: Alterações no layout, header e footer, podem ser testadas individualmente, reduzindo falsos positivos e facilitando o controle de regressões.
- Testes mais rápidos e específicos: O `Layout.test.jsx` passa a verificar a presença e integração dos subcomponentes, enquanto os testes `Header.test.jsx` e `Footer.test.jsx` validam de forma isolada seus respectivos conteúdos e comportamentos internos.

Essa abordagem reforça a consistência entre a arquitetura da aplicação e a estratégia de testes.

7.2. Teste Header.test.jsx:

O componente `src/tests/components/Header.test.jsx` é responsável por renderizar o cabeçalho da aplicação, contendo o título da pousada.

```
import { render, screen } from "@testing-library/react";
import Header from "../../components/Header";

describe("Header", () => {
  test("deve renderizar o título da pousada", () => {
    render(<Header />);
    expect(screen.getByText("Pousada Pedra Furada")).toBeInTheDocument();
  });
});
```

O teste deve localizar o texto "Pousada Pedra Furada" dentro do documento e caso o nome do estabelecimento seja alterado futuramente, este teste apontará a divergência imediatamente.

7.3. Teste Footer.test.jsx:

O `Footer.jsx` exibe informações institucionais fixas da aplicação, como os direitos autorais da pousada.

O teste abaixo, que deve ficar no caminho `src/tests/components/Footer.test.jsx`, garante que essas informações estejam corretamente renderizadas.

```
import { render, screen } from "@testing-library/react";
import Footer from "../../components/Footer";
describe("Footer", () => {
  test("deve exibir o texto de direitos autorais", () => {
    render(<Footer />);
    expect(
      screen.getByText("© 2025 Pousada Pedra Furada")
    ).toBeInTheDocument();
  });
});
```

O teste deve confirmar que o rodapé contém a mensagem de copyright e se houver alterações de ano ou inclusão de novos elementos (como links ou redes sociais), o teste deverá ser atualizado de acordo.

7.4. Ajustando o Layout.test.jsx:

Após a separação dos componentes, o teste do Layout foi simplificado para validar apenas a integração entre as partes da aplicação. Ele garante que o Layout renderize corretamente o cabeçalho (Header), o conteúdo dinâmico (children) e o rodapé (Footer) dentro de sua estrutura principal, sem verificar o conteúdo interno desses componentes, assegurando que:

- O Header seja efetivamente chamado e renderizado na posição superior da página.
- O conteúdo passado via children seja exibido dentro da área principal (<main>).
- O Footer seja renderizado na parte inferior.

```
// src/tests/components/Layout.test.jsx
import { render, screen } from "@testing-library/react";
import Layout from "../components/Layout";
// Mocka Header e Footer para isolar o Layout
jest.mock("../components/Header", () => () => <header>Mock Header</header>);
jest.mock("../components/Footer", () => () => <footer>Mock Footer</footer>);
describe("Layout.jsx — Estrutura principal da aplicação", () => {
  it("deve renderizar Header, conteúdo filho e Footer", () => {
    render(<Layout><p>Conteúdo interno</p></Layout>);
    // Verifica se o Header mockado foi renderizado
    expect(screen.getByText("Mock Header")).toBeInTheDocument();
    // Verifica o conteúdo filho (children)
    expect(screen.getByText("Conteúdo interno")).toBeInTheDocument();
    // Verifica se o Footer mockado foi renderizado
    expect(screen.getByText("Mock Footer")).toBeInTheDocument();
  });
});
```

Esse teste atua como uma verificação de integração mínima, assegurando que o Layout mantenha a estrutura base da aplicação corretamente montada, mesmo que os componentes internos (Header e Footer) passem por alterações independentes de conteúdo, estilo ou lógica.

```
EXPLORER
PEDRA-FURADA [CODESPACES: BOOKIS...
> __mocks__
> .github
> .next
> node_modules
> prisma
> public
v src
  v components
    Footer.jsx
    Header.jsx
    Layout.jsx
  > hooks
  > lib
  v pages
    > api
    index.jsx
    LoginPage.jsx
    RelatoriosPage.jsx
    TarefasPage.jsx
  > styles
  v tests
    v components
      Footer.test.jsx
      Header.test.jsx
    > pages
    App.test.jsx
    index.test.jsx
    Layout.test.jsx
    main.test.jsx
  App.jsx
  main.jsx
  .env.example
  .gitignore
  babel.config.js
  jest.config.js
  jest.setup.js
> OUTLINE
```

Layout.test.jsx | Layout.jsx | index.jsx | babel.config.js | Header.test.jsx

src > tests > Layout.test.jsx > describe("Layout.jsx — Estrutura principal da aplicação") callback > it("deve re

```jsx
// src/tests/components/Layout.test.jsx
import { render, screen } from "@testing-library/react";
import Layout from "../components/Layout";

// Mocka Header e Footer para isolar o Layout
jest.mock("../components/Header", () => () => <header>Mock Header</header>);
jest.mock("../components/Footer", () => () => <footer>Mock Footer</footer>);

describe("Layout.jsx — Estrutura principal da aplicação", () => {
  it("deve renderizar Header, conteúdo filho e Footer", () => {
    render(
      <Layout>
        <p>Conteúdo interno</p>
      </Layout>
    );

    // Verifica se o Header mockado foi renderizado
    expect(screen.getByText("Mock Header")).toBeInTheDocument();

    // Verifica o conteúdo filho (children)
    expect(screen.getByText("Conteúdo interno")).toBeInTheDocument();

    // Verifica se o Footer mockado foi renderizado
    expect(screen.getByText("Mock Footer")).toBeInTheDocument();
  });
});
```

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS | npm - pedra-furada

```
PASS  src/tests/Layout.test.jsx
PASS  src/tests/App.test.jsx
PASS  src/tests/index.test.jsx
PASS  src/tests/components/Header.test.jsx
PASS  src/tests/components/Footer.test.jsx
PASS  src/tests/main.test.jsx

Test Suites: 6 passed, 6 total
Tests:       10 passed, 10 total
Snapshots:   0 total
Time:        1.513 s
Ran all test suites.
```

Proceder com o commit: “test: Criação dos testes Layout.test.jsx e Footer.test.jsx e ajuste do teste Layout.test.jsx.”.